Chapter 3. Deploying a Kubernetes Cluster

Now that you have successfully built an application container, the next step is to learn how to transform it into a complete, reliable, scalable distributed system. To do that, you need a working Kubernetes cluster. At this point, there are cloud-based Kubernetes services in most public clouds that make it easy to create a cluster with a few command-line instructions. We highly recommend this approach if you are just getting started with Kubernetes. Even if you are ultimately planning on running Kubernetes on bare metal, it's a good way to quickly get started with Kubernetes, learn about Kubernetes itself, and then learn how to install it on physical machines. Furthermore, managing a Kubernetes cluster is a complicated task in itself, and, for most people, it makes sense to defer this management to the cloud—especially when the management service is free in most clouds.

Of course, using a cloud-based solution requires paying for those cloud-based resources as well as having an active network connection to the cloud. For these reasons, local development can be more attractive, and in that case, the `minikube` tool provides an easy-to-use way to get a local Kubernetes cluster up and running in a VM on your local laptop or desktop. Though this is a nice option, `minikube` only creates a single-node cluster, which doesn't quite demonstrate all of the aspects of a complete Kubernetes cluster. For that reason, we recommend people start with a cloud-based solution, unless it really doesn't work for their situation. A more recent alternative is to run a Docker-in-Docker cluster, which can spin up a multinode cluster on a single machine. This project is still in beta, though, so keep in mind that you may encounter unexpected issues.

If you truly insist on starting on bare metal, see the Appendix at the end of this book for instructions for building a cluster from a collection of Raspberry Pi single-board computers. These instructions use the `kubeadm` tool and can be adapted to other machines beyond Raspberry Pis.

Installing Kubernetes on a Public Cloud

Provider

This chapter covers installing Kubernetes on the three major cloud providers: the Google Cloud Platform, Microsoft Azure, and Amazon Web Services.

If you choose to use a cloud provider to manage Kubernetes, you need to install only one of these options; once you have a cluster configured and ready to go, you can skip to "The Kubernetes Client", unless you would prefer to install Kubernetes elsewhere.

Installing Kubernetes with Google Kubernetes Engine

The Google Cloud Platform (GCP) offers a hosted Kubernetes-as-a-Service called Google Kubernetes Engine (GKE). To get started with GKE, you need a Google Cloud Platform account with billing enabled and the `gcloud` tool installed.

Once you have `gcloud` installed, set a default zone:

```
$ gcloud config set compute/zone us-west1-a
```

Then you can create a cluster:

```
$ gcloud container clusters create kuar-cluster --num-nodes=3
```

This will take a few minutes. When the cluster is ready, you can get credentials for the cluster using:

```
$ gcloud container clusters get-credentials kuar-cluster
```

If you run into trouble, you can find the complete instructions for creating a GKE cluster in the Google Cloud Platform documentation.

Installing Kubernetes with Azure Kubernetes Service

Microsoft Azure offers a hosted Kubernetes-as-a-Service as part of the Azure Container Service. The easiest way to get started with Azure Container Service is to use the built-in Azure Cloud Shell in the Azure portal. You can activate the shell by clicking the shell icon in the upper-right toolbar:



The shell has the `az` tool automatically installed and configured to work with your Azure environment.

Alternatively, you can install the `az` CLI on your local machine.

When you have the shell up and working, you can run:

```
$ az group create --name=kuar --location=westus
```

Once the resource group is created, you can create a cluster using:

```
$ az aks create --resource-group=kuar --name=kuar-cluster
```

This will take a few minutes. Once the cluster is created, you can get credentials for the cluster with:

```
$ az aks get-credentials --resource-group=kuar --name=kuar-cluster
```

If you don't already have the `kubectl` tool installed, you can install it using:

```
$ az aks install-cli
```

You can find complete instructions for installing Kubernetes on Azure in the Azure documentation.

Installing Kubernetes on Amazon Web Services

Amazon offers a managed Kubernetes service called Elastic Kubernetes Service (EKS). The easiest way to create an EKS cluster is via the open source `eksctl` command-line tool.

Once you have `eksctl` installed and in your path, you can run the following command to create a cluster:

```
$ eksctl create cluster
```

For more details on installation options (such as node size and more), view the help using this command:

```
$ eksctl create cluster --help
```

The cluster installation includes the right configuration for the `kubectl` command-line tool. If you don't already have `kubectl` installed, follow the instructions in the documentation.

Installing Kubernetes Locally Using minikube

If you need a local development experience, or you don't want to pay for cloud resources, you can install a simple single-node cluster using `minikube`. Alternatively, if you have already installed Docker Desktop, it comes bundled with a single-machine installation of Kubernetes.

While `minikube` (or Docker Desktop) is a good simulation of a Kubernetes cluster, it's really intended for local development, learning, and experimentation. Because it only runs in a VM on a single node, it doesn't provide the reliability of a distributed Kubernetes cluster. In addition, certain features described in this book require integration with a cloud provider. These features are either not available or work in a limited way with `minikube`.

NOTE

You need to have a hypervisor installed on your machine to use `minikube`. For Linux and macOS, this is generally VirtualBox. On Windows, the Hyper-V hypervisor is the default option. Make sure you install the hypervisor before using `minikube`.

You can find the `minikube` tool on GitHub. There are binaries for Linux, macOS, and Windows that you can download. Once you have the `minikube` tool installed, you can create a local cluster using:

```
$ minikube start
```

This will create a local VM, provision Kubernetes, and create a local `kubectl` configuration that points to that cluster. As mentioned previously, this cluster only has a single node, so while it is useful, it has some differences with most production deployments of Kubernetes.

When you are done with your cluster, you can stop the VM with:

```
$ minikube stop
```

If you want to remove the cluster, you can run:

```
$ minikube delete
```

Running Kubernetes in Docker

A different approach to running a Kubernetes cluster, which has been developed more recently, uses Docker containers to simulate multiple Kubernetes nodes instead of running everything in a virtual machine. The kind project provides a great experience for launching and managing test clusters in Docker. (*kind* stands for Kubernetes IN Docker.) kind is still a work in progress (pre 1.0), but is widely used by those building Kubernetes for fast and easy testing.

Installation instructions for your platform can be found at the kind site. Once you get it installed, creating a cluster is as easy as:

```
$ kind create cluster --wait 5m
$ export KUBECONFIG="$(kind get kubeconfig-path)"
$ kubectl cluster-info
$ kind delete cluster
```

The Kubernetes Client

The official Kubernetes client is `kubectl`: a command-line tool for interacting with the Kubernetes API. `kubectl` can be used to manage most Kubernetes objects, such as Pods, ReplicaSets, and Services. `kubectl` can also be used to explore and verify the overall health of the cluster.

We'll use the `kubectl` tool to explore the cluster you just created.

Checking Cluster Status

The first thing you can do is check the version of the cluster that you are running:

```
$ kubectl version
```

This will display two different versions: the version of the local `kubectl` tool, as well as the version of the Kubernetes API server.

NOTE

Don't worry if these versions are different. The Kubernetes tools are backward- and forward-compatible with different versions of the Kubernetes API as long as you stay within two minor versions for both the tools and the cluster and don't try to use newer features on an older cluster. Kubernetes follows the semantic versioning specification, where the minor version is the middle number (e.g., the 18 in 1.18.2). However, you will want to make sure that you are within the supported version skew, which is three versions. If you are not, you may run into problems.

Now that we've established that you can communicate with your Kubernetes cluster, we'll explore the cluster in more depth.

First, you can get a simple diagnostic for the cluster. This is a good way to verify that your cluster is generally healthy:

```
$ kubectl get componentstatuses
```

The output should look like this:

```
NAME                 STATUS    MESSAGE             ERROR
scheduler            Healthy   ok
controller-manager   Healthy   ok
etcd-0               Healthy   {"health": "true"}
```

NOTE

As Kubernetes changes and improves over time, the output of the `kubectl` command sometimes changes. Don't worry if the output doesn't look exactly identical to what is shown in the examples in this book.

You can see here the components that make up the Kubernetes cluster. The `controller-manager` is responsible for running various controllers that regulate behavior in the cluster; for example, ensuring that all of the replicas of a service are available and healthy. The `scheduler` is responsible for placing different Pods onto different nodes in the cluster. Finally, the `etcd` server is the storage for the cluster where all of the API objects are stored.

Listing Kubernetes Nodes

Next, you can list out all of the nodes in your cluster:

```
$ kubectl get nodes
NAME    STATUS   ROLES                  AGE    VERSION
kube0   Ready    control-plane,master   45d    v1.22.4
kube1   Ready    <none>                 45d    v1.22.4
kube2   Ready    <none>                 45d    v1.22.4
kube3   Ready    <none>                 45d    v1.22.4
```

You can see this is a four-node cluster that's been up for 45 days. In Kubernetes, nodes are separated into `control-plane` nodes that contain containers like the API server, scheduler, etc., which manage the cluster, and `worker` nodes where your containers will run. Kubernetes won't generally schedule work onto `control-plane` nodes to ensure that user workloads don't harm the overall operation of the cluster.

You can use the `kubectl describe` command to get more information about a specific node, such as `kube1`:

```
$ kubectl describe nodes kube1
```

First, you see basic information about the node:

```
Name:               kube1
Role:
```

```
Labels:             beta.kubernetes.io/arch=arm
                    beta.kubernetes.io/os=linux
                    kubernetes.io/hostname=node-1
```

You can see that this node is running the Linux OS on an ARM processor.

Next, you see information about the operation of `kube1` itself (dates have been removed from this output for concision):

```
Conditions:
  Type             Status  ...  Reason                  Message
 -----             ------      ------                  -------
  NetworkUnavailable  False   ...  FlannelIsUp             Flannel...
  MemoryPressure      False   ...  KubeletHasSufficientMemory  kubelet...
  DiskPressure        False   ...  KubeletHasNoDiskPressure    kubelet...
  PIDPressure         False   ...  KubeletHasSufficientPID     kubelet...
  Ready               True    ...  KubeletReady            kubelet...
```

These statuses show that the node has sufficient disk and memory space and is reporting that it is healthy to the Kubernetes master. Next, there is information about the capacity of the machine:

```
Capacity:
 alpha.kubernetes.io/nvidia-gpu:  0
 cpu:                             4
 memory:                          882636Ki
 pods:                            110
Allocatable:
 alpha.kubernetes.io/nvidia-gpu:  0
 cpu:                             4
 memory:                          882636Ki
 pods:                            110
```

Then there is information about the software on the node, including the version of Docker that is running, the versions of Kubernetes and the Linux kernel, and more:

```
System Info:
  Machine ID:             44d8f5dd42304af6acde62d233194cc6
  System UUID:            c8ab697e-fc7e-28a2-7621-94c691120fb9
  Boot ID:                e78d015d-81c2-4876-ba96-106a82da263e
  Kernel Version:         4.19.0-18-amd64
```

```
  OS Image:                   Debian GNU/Linux 10 (buster)
  Operating System:           linux
  Architecture:               amd64
  Container Runtime Version:  containerd://1.4.12
  Kubelet Version:            v1.22.4
  Kube-Proxy Version:         v1.22.4
PodCIDR:                      10.244.1.0/24
PodCIDRs:                     10.244.1.0/24
```

Finally, there is information about the Pods that are currently running on this node:

```
Non-terminated Pods:          (3 in total)
  Namespace   Name       CPU Requests CPU Limits Memory Requests Memory Limits
  ---------   ----       ------------ ---------- --------------- -------------
  kube-system kube-dns...  260m (6%)    0 (0%)     140Mi (16%)     220Mi (25%)
  kube-system kube-fla...  0 (0%)       0 (0%)     0 (0%)          0 (0%)
  kube-system kube-pro...  0 (0%)       0 (0%)     0 (0%)          0 (0%)
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.
  CPU Requests  CPU Limits      Memory Requests Memory Limits
  ------------  ----------      --------------- -------------
  260m (6%)     0 (0%)          140Mi (16%)     220Mi (25%)
No events.
```

From this output, you can see the Pods on the node (e.g., the `kube-dns` Pod that supplies DNS services for the cluster), the CPU and memory that each Pod is requesting from the node, as well as the total resources requested. It's worth noting here that Kubernetes tracks both the *requests* and the upper *limits* for resources for each Pod that runs on a machine. The difference between requests and limits is described in detail in Chapter 5, but in a nutshell, resources requested by a Pod are guaranteed to be present on the node, while a Pod's limit is the maximum amount of a given resource that a Pod can consume. A Pod's limit can be higher than its request, in which case the extra resources are supplied on a best-effort basis. They are not guaranteed to be present on the node.

Cluster Components

One of the interesting aspects of Kubernetes is that many of the components that make up the Kubernetes cluster are actually deployed using Kubernetes itself. We'll take a

look at a few of these. These components use a number of the concepts that we'll introduce in later chapters. All of these components run in the `kube-system` namespace.[1]

Kubernetes Proxy

The Kubernetes proxy is responsible for routing network traffic to load-balanced services in the Kubernetes cluster. To do its job, the proxy must be present on every node in the cluster. Kubernetes has an API object named DaemonSet, which you will learn about in Chapter 11, that is used in many clusters to accomplish this. If your cluster runs the Kubernetes proxy with a DaemonSet, you can see the proxies by running:

```
$ kubectl get daemonSets --namespace=kube-system kube-proxy
NAME       DESIRED   CURRENT   READY   UP-TO-DATE   AVAILABLE   NODE SELECTOR
kube-proxy  5         5         5       5            5           ...  45d
```

Depending on how your cluster is set up, the DaemonSet for the `kube-proxy` may be named something else, or it's possible that it won't use a DaemonSet at all. Regardless, the `kube-proxy` container should be running on all nodes in a cluster.

Kubernetes DNS

Kubernetes also runs a DNS server, which provides naming and discovery for the services that are defined in the cluster. This DNS server also runs as a replicated service on the cluster. Depending on the size of your cluster, you may see one or more DNS servers running in your cluster. The DNS service is run as a Kubernetes deployment, which manages these replicas (this may also be named `coredns` or some other variant):

```
$ kubectl get deployments --namespace=kube-system core-dns
NAME      DESIRED   CURRENT   UP-TO-DATE   AVAILABLE   AGE
core-dns  1         1         1            1           45d
```

There is also a Kubernetes service that performs load balancing for the DNS server:

```
$ kubectl get services --namespace=kube-system core-dns
NAME      CLUSTER-IP   EXTERNAL-IP   PORT(S)         AGE
core-dns  10.96.0.10   <none>        53/UDP,53/TCP   45d
```

This shows that the DNS service for the cluster has the address 10.96.0.10. If you log in to a container in the cluster, you'll see that this has been populated into the */etc/resolv.conf* file for the container.

Kubernetes UI

If you want to visualize your cluster in a graphical user interface, most of the cloud providers integrate such a visualization into the GUI for their cloud. If your cloud provider doesn't provide such a UI, or you prefer an in-cluster GUI, there is a community supported GUI that you can install. See the documentation on how to install the dashboard for these clusters. You can also use extensions for development environments like Visual Studio Code to see the state of your cluster at a glance.

Summary

Hopefully at this point you have a Kubernetes cluster (or three) up and running and you've used a few commands to explore the cluster you have created. Next, we'll spend some more time exploring the CLI to that Kubernetes cluster and teach you how to master the `kubectl` tool. Throughout the rest of the book, you'll be using `kubectl` and your test cluster to explore the various objects in the Kubernetes API.

1 As you'll learn in the next chapter, a namespace in Kubernetes is an entity for organizing Kubernetes resources. You can think of it like a folder in a filesystem.

Previous chapter
< 2. Creating and Running Containers

Next chapter
4. Common kubectl Commands >